

数据库工程作业

要求:

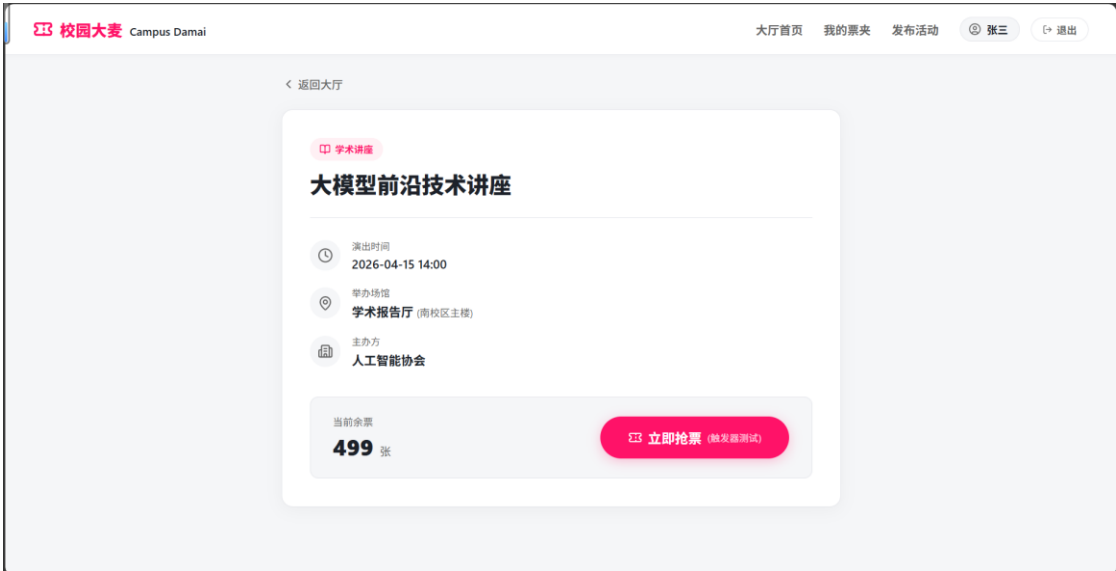
- 1. 完成一个小型的数据库信息管理系统（或部分功能），并填写工程作业报告；程序和报告请在规定时间之内上传。
- 2. 开发模式（B/S 或 C/S）、开发高级语言任选，后台数据库使用大型数据库管理系统（SQL Server、Oracle、MySQL 等），不要使用桌面数据库。
- 3. 报告中所列举的四种操作，每种操作举一个例子即可。
- 4. 作业成绩按照报告中的标准评分，程序只实现报告中涉及的部分即可。
- 5. 作业完成后，请将工程作业报告和程序打包提交给助教老师，并联系助教老师进行系统说明和演示，回答相关问题。

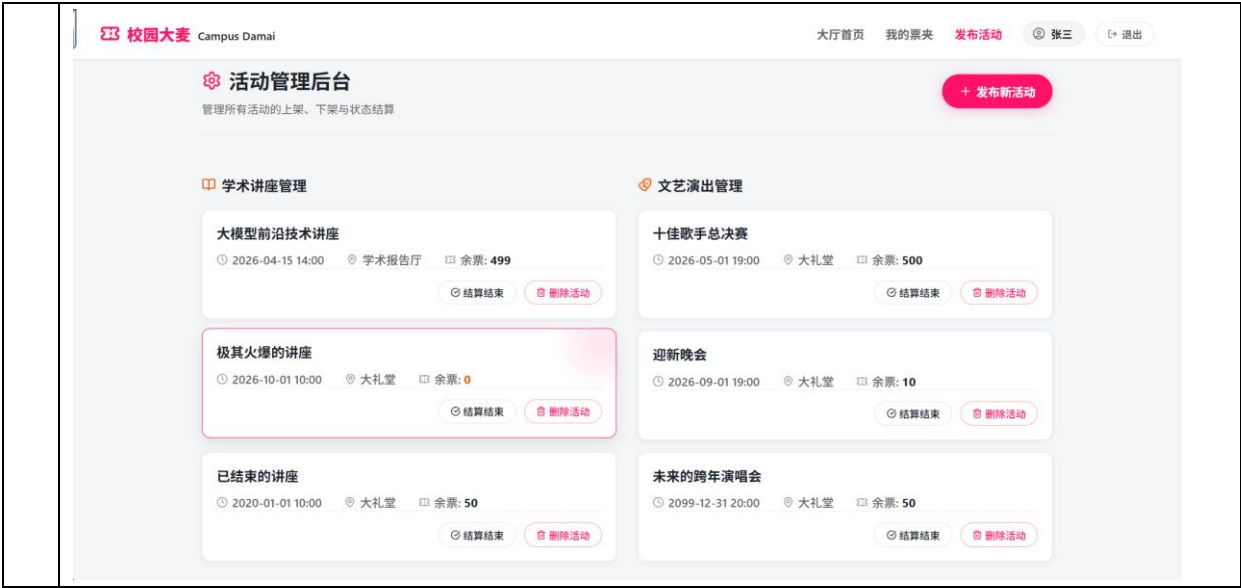
工程作业报告

1. 项目信息（10 分）

学号	2411033	姓名	季艾佳	专业	计算机科学与技术
项目名称	校园“大麦”				
必备环境	前端语言环境：TypeScript ~6.0.0 前端编译构建工具：Vite v8.0.14 包管理与工程依赖驱动：npm v11.9.0 后端核心技术栈：Java 21 (JDK 21) / Spring Boot v2.7.18 数据仓库组件：MySQL 8.0 社区版				
系统主要功能简介（4分）	实现了一个校园活动信息管理平台，实现了活动的增删改查、买票、退票、后台管理等功能。包括但不限于以下代表功能： 含有事务应用的删除操作：基于订单状态处理退票请求。 触发器控制下的添加操作：票务购买交易处理并添加到订单 存储过程控制下的更新操作：活动结束后后台处理统一将票务变成已完成操作 含有视图的查询操作：平台首页以及活动详情页显示组织方以及活动场馆				

系统主要页面截图（6分）





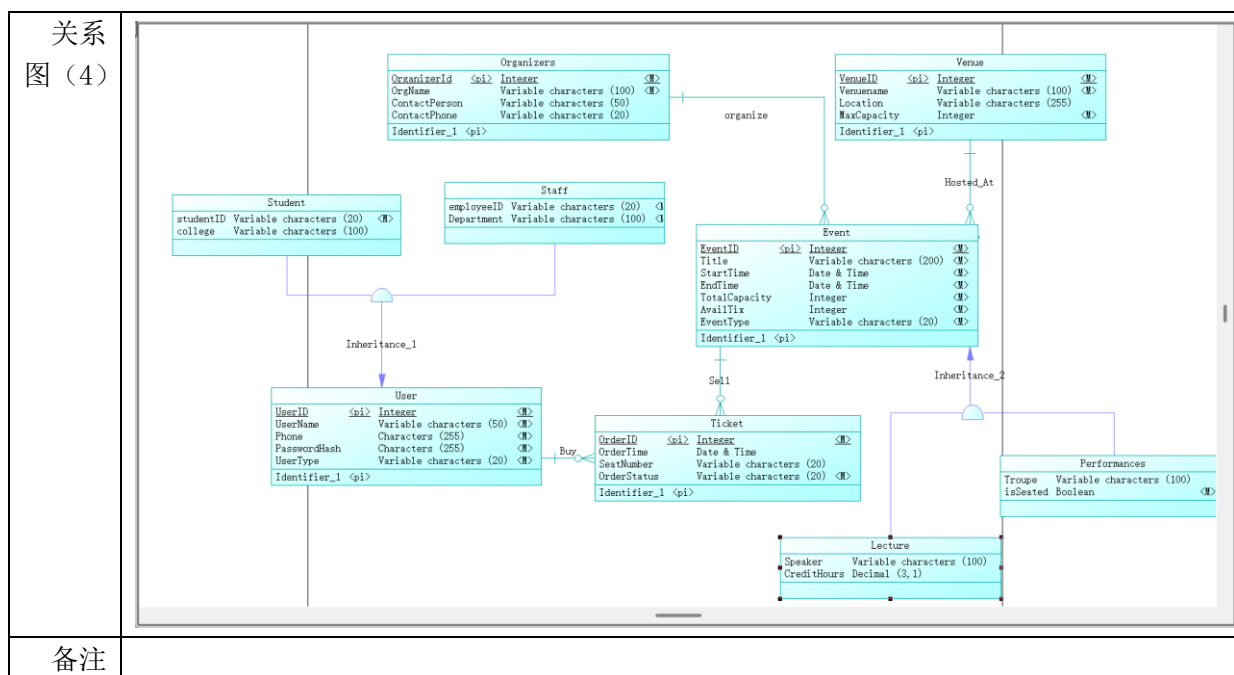
2. 系统配置（10 分）

说明		(2 分) 请说明系统配置情况（后台数据库，高级语言）； (8 分) 请使用连接串连接高级语言和数据库，并分析字符串的各个部分。			
配置 步骤 2 分	DBMS	1. 下载并安装 MySQL 8.0 服务端，以及 mysqlworkbench，设置运行端口（默认 3306）。			
	高级语言	2. 由 powerdesigner 建立 E-R 模型，一键导出 sql 建表，并编写触发器、存储过程等			
		1. 配置 spring-boot 后端并与数据库建立联系			
		2. 配置 vue 环境			
连接串 分析 (6 分)		序号	名称	功能说明	取值
		1	协议主名称	指定 Java 数据库连接的标准 API 协议类型	jdbc
		2	数据库子协议	指定要加载和连接的特定数据库驱动类型（此处为 MySQL 驱动）	mysql
		3	数据库主机地址	指定 MySQL 数据库服务器所在的网络 IP 地址（localhost 代表本地回环测试地址）	localhost
		4	网络监听端口	MySQL 数据库服务在操作系统中开放并监听的默认网络通信端口	3306
		5	物理数据库名	本次连接指定要访问和绑定的具体数据库实例名称	unitix
		6	useUnicode	是否使用统一码（Unicode）字符集。在处理多语言、特殊符号或特定的排版字符时激活，防止乱码	true
		7	characterEncoding	强制指定数据库传输时的字符编码格式，确保前后端数据交换时中文不出现乱码	utf8
		8	useSSL	安全套接层协议开关。本地开发环境关闭加密传输，	false

			从而提升数据库连接建立的响应速度	
	9	server Timezone	全局时区强制声明。确保 Java 运行时与 MySQL 物理机时区（东八区）对齐，防止购票或结算时间出现错位	Asia/Shanghai
连接串代码 （截屏） （2 分）	<pre>spring: datasource: url: jdbc:mysql://localhost:3306/unitix?useUnicode=true&characterEncoding=utf8&useSSL=false&serverTimezone=Asia/Shanghai username: root password: 123456 driver-class-name: com.mysql.cj.jdbc.Driver</pre>			
备注				

3. 数据库设计（14 分）

说明	（10 分）按照数据表的创建顺序，依次给出所涉及数据表的信息，其中参照字段以“（字段 1，字段 2，……，字段 n）”的形式给出，被参照字段以“表名（字段 1，字段 2，……，字段 n）”的形式给出； （4 分）一般 DBMS 都可以为数据库生成关系图，请将该图片截屏并粘贴到表格中。				
数据表 （10）	创建顺序	数据表名称	主键	参照属性	被参照表及属性
	1	Organizers	OrganizerId	无	无
	2	Venue	VenueID	无	无
	3	User	UserID	无	无
	4	Event	EventID	(OrganizerId)	Organizers (OrganizerId)
				(VenueID)	Venue (VenueID)
	5	Lecture	EventID	(EventID)	Event (EventID)
	6	Performances	EventID	(EventID)	Event (EventID)
	7	Student	UserID	(UserID)	User (UserID)
	8	Staff	UserID	(UserID)	User (UserID)
	9	Ticket	OrderID	(EventID)	Event (EventID)
				(UserID)	User (UserID)



4. 含有事务应用的删除操作 (13 分)

说明	(1 分) 简要说明该操作所要完成的功能; (2 分) 该操作会涉及的表 (必须含有两张或两张以上的关系表, 同时以“表名”的形式给出) (1 分) 表连接涉及字段描述 (描述方式为“表 1. 属性=表 2. 属性”) (1 分) 删除条件涉及的字段描述 (以“表名. 属性=? ”形式给出) (4 分) 实现该操作的关键代码 (高级语言、SQL), 截图即可; (其中如果删除语句中不包含任何形式的事务应用将扣除 3 分) (4 分) 如何执行该操作, 按所述方法能够正常演示程序则给分。
功能描述 (1 分)	实现校园活动管理系统中的“用户退票 (取消订单)”功能。 仅允许状态为“已支付 (Paid)”且演出尚未结束的订单进行退票 申请退票时, 要将 ticket 表中的记录删除, 同时将 event 中剩余票数 AvailTix 加 1。
涉及的表 (2 分)	Ticket, Event
表连	Ticket.EventID = Event.EventID

接 涉 及 字 段 （ 1 分 ）		
删 除 条 件 字 段 描 述 （ 1 分 ）	字段	规则
	Ticket.OrderID = ?	根据前端传来的订单号来删除对应元素

代码
(
4
分
)

```
63 -- 含有事务应用的删除操作
64 DELIMITER //
65 • -- 创建包含状态校验与时间校验的退票存储过程
66 CREATE PROCEDURE proc_refund_ticket(
67     IN p_OrderID INT,
68     OUT p_Status VARCHAR(20)
69 )
70 BEGIN
71     DECLARE v_EventID INT;
72     DECLARE v_OrderStatus VARCHAR(20);
73     DECLARE v_EndTime DATETIME;
74     DECLARE EXIT HANDLER FOR SQLEXCEPTION
75     BEGIN
76         ROLLBACK;
77         SET p_Status = 'System Error';
78     END;
79     START TRANSACTION;
80     SELECT EventID, OrderStatus INTO v_EventID, v_OrderStatus
81     FROM Ticket
82     WHERE OrderID = p_OrderID
83     FOR UPDATE;
84     IF v_EventID IS NULL THEN -- 订单不存在
85         ROLLBACK;
86         SET p_Status = 'Order Not Found';
87     ELSEIF v_OrderStatus <> 'Paid' THEN -- 订单状态是不是“已支付”
88         ROLLBACK;
89         SET p_Status = 'Wrong Status';
90     ELSE
91         SELECT EndTime INTO v_EndTime -- 查询该活动的结束时间
92         FROM Event
93         WHERE EventID = v_EventID;
94         IF v_EndTime <= NOW() THEN -- 判断活动是否已经结束
95             ROLLBACK;
96             SET p_Status = 'Event Ended';
97         ELSE
98             -- 执行退票流程
99             DELETE FROM Ticket WHERE OrderID = p_OrderID;
100             UPDATE Event SET AvailTix = AvailTix + 1 WHERE EventID = v_EventID;
101             COMMIT;
102             SET p_Status = 'Refund Success';
103         END IF;
104     END IF;
105 END //
106 DELIMITER ;
```

高级语言与前端实现:

1. 前端发起:

```
const res = await axios.post(`http://localhost:8080/ticket/refund?orderId=${ticketId}`)
```

2. 后端 control 接口接收:

```
// 退票接口
// 前端访问: POST http://localhost:8080/ticket/refund
@PostMapping("/refund")
public Result<String> refundTicket(@RequestParam Integer orderId) {
    String status = ticketService.refundTicket(orderId);

    if ("Refund Success".equals(status)) {
        return Result.success("退票成功! 票款将原路返回。");
    } else if ("Event Ended".equals(status)) {
        return Result.error("退票失败: 该演出已经结束!");
    } else if ("Wrong Status".equals(status)) {
        return Result.error("退票失败: 订单状态异常(可能未支付或已退款)!");
    } else if ("Order Not Found".equals(status)) {
        return Result.error("退票失败: 找不到该订单!");
    } else {
        return Result.error("系统异常: " + status);
    }
}
```

3. 后端 server 处理:

```
// 退票业务
String refundTicket(Integer orderId);

@Override
public String refundTicket(Integer orderId) {
    Map<String, Object> params = new HashMap<>();
    params.put("orderId", orderId);

    ticketMapper.refundTicket(params);

    return (String) params.get("status");
}
```

4. 后端 mapper 调用:

```
// 调用退票存储过程
@Select("CALL proc_refund_ticket(" +
    "#{orderId, mode=IN, jdbcType=INTEGER}, " +
    "#{status, mode=OUT, jdbcType=VARCHAR})")
@Options(statementType = StatementType.CALLABLE)
void refundTicket(Map<String, Object> params);
```

程序演示 (4 分)



5. 触发器控制下的添加操作（20 分）

说明	(1 分) 简要说明该操作所要完成的功能; (2 分) 简要说明该触发器所要完成的功能 (1 分) 该操作会涉及的表 (以 “表名” 的形式给出)。 (2 分) 该操作输入数据以及输入数据应该满足的条件, 如: 数值范围、是否为空; (6 分) 实现该操作的关键代码 (高级语言、SQL), 截图即可; (8 分) 如何执行该操作, 按所述方法能够正常演示程序则给分。	
功能描述 (1 分)	用户购买门票时, 在 ticket 表中添加一条购票记录 (在对应活动有余票的前提下)。	
触发器描述 (2 分)	trg_: 在插入表之前触发, 首先看是否有余票, 没有的话抛出系统异常。 有余票后 event 表中的 availtix 减一。	
涉及的表 (1 分)	Ticket, Event	
输入数据 (2 分)	字段	规则
	Ticket.EventID	非空, 传入的 ID 在 Event 表中对应的当前 AvailTix > 0。
	Ticket.UserID	非空, 且有效
	Ticket.OrderStatus	非空, 一般要是 'Paid'

插入操作源码 (3分)

```
22 DELIMITER //
23 • CREATE PROCEDURE proc_buy_ticket(
24     IN p_UserID INT,
25     IN p_EventID INT,
26     OUT p_Status VARCHAR(20)
27 )
28 BEGIN
29     DECLARE v_AvailTix INT;
30     START TRANSACTION;
31     SELECT AvailTix INTO v_AvailTix
32     FROM Event
33     WHERE EventID = p_EventID
34     FOR UPDATE;
35     IF v_AvailTix > 0 THEN
36         INSERT INTO Ticket (OrderTime, OrderStatus, EventID, UserID)
37         VALUES (NOW(), 'Paid', p_EventID, p_UserID);
38         COMMIT;
39         SET p_Status = 'Success';
40     ELSE
41         ROLLBACK;
42         SET p_Status = 'Sold Out';
43     END IF;
44 END //
45 DELIMITER ;
```

触发器源码 (3分)

```
1  -- 触发器控制下的添加操作
2  DELIMITER //
3  • CREATE TRIGGER trg
4  BEFORE INSERT ON Ticket
5  FOR EACH ROW
6  BEGIN
7      DECLARE current_avail INT;
8      SELECT AvailTix INTO current_avail
9      FROM Event
10     WHERE EventID = NEW.EventID;
11     IF current_avail <= 0 THEN -- 没票了抛出异常
12         SIGNAL SQLSTATE '45000'
13         SET MESSAGE_TEXT = '触发器拦截：该活动已售罄！';
14     ELSE
15         UPDATE Event -- 有票
16         SET AvailTix = AvailTix - 1
17         WHERE EventID = NEW.EventID;
18     END IF;
19 END //
20 DELIMITER ;
```

高级语言与前端实现：

1. 前端发起：

	<pre>const response = await axios.post('http://localhost:8080/ticket/buy', null, { params: { userId: currentUserId, eventId: eventId } })</pre> 2. 后端 control 接口接收: <pre>// 购票接口 // 前端访问: POST http://localhost:8080/ticket/buy @PostMapping("/buy") public Result<String> buyTicket(@RequestParam Integer userId, @RequestParam Integer eventId) { // 调用 Service 层拿到存储过程返回的状态 String status = ticketService.buyTicket(userId, eventId); if ("Success".equals(status)) { return Result.success("购票成功! 您的订单已生成。"); } else if ("Sold Out".equals(status)) { return Result.error("手慢啦, 该活动门票已售罄!"); } else { return Result.error("购票失败, 系统异常: " + status); } }</pre> 3. 后端 server 处理: <pre>@Override public String buyTicket(Integer userId, Integer eventId) { Map<String, Object> params = new HashMap<>(); params.put("userId", userId); params.put("eventId", eventId); ticketMapper.buyTicket(params); return (String) params.get("status"); }</pre> 4. 后端 mapper 调用: <pre>@Select("CALL proc_buy_ticket(" + "#{userId, mode=IN, jdbcType=INTEGER}, " + "#{eventId, mode=IN, jdbcType=INTEGER}, " + "#{status, mode=OUT, jdbcType=VARCHAR})") @Options(statementType = StatementType.CALLABLE) void buyTicket(Map<String, Object> params);</pre>
程序演示 (4分)	说明: 不违背触发器能够执行插入操作。  The screenshot shows a web application interface. At the top, a dark modal dialog box is displayed with the text 'localhost:5173 显示' and '确定要购买这张票吗?' (Are you sure you want to buy this ticket?). Below the dialog, the main content area shows a lecture event titled '极其火爆的讲座' (Extremely Popular Lecture). The event details include: '演出时间: 2026-10-01 10:00' (Performance Time: 2026-10-01 10:00), '举办场馆: 大礼堂 (北校区)' (Venue: Grand Auditorium (North Campus)), and '主办方: 校学生会' (Organizer: Student Union). At the bottom, it shows '当前余票: 1 张' (Current remaining tickets: 1 ticket) and a red button labeled '立即抢票 (模拟测试)' (Grab ticket immediately (simulation test)).

	localhost:5173 显示 抢票成功! 去【我的票夹】看看吧。 确定 极其火爆的讲座 演出时间 2026-10-01 10:00 举办场馆 大礼堂 (北校区) 主办方 校学生会 当前余票 1 张 立即抢票 (触发器测试) 我的票夹 管理您的所有购票订单 未支付 已支付 已结束 极其火爆的讲座 已购票 2026-10-01 10:00 大礼堂 订单编号: #23 退票 / 删除
程序演示 (4分)	说明: 违背触发器要求, 不能够执行插入操作, 系统报错。 localhost:5173 显示 抢票失败: 手慢啦, 该活动门票已售罄! 确定 极其火爆的讲座 演出时间 2026-10-01 10:00 举办场馆 大礼堂 (北校区) 主办方 校学生会 当前余票 0 张 立即抢票 (触发器测试)
备注	

6. 存储过程控制下的更新操作（18分）

说明	(1分) 简要说明该操作所要完成的功能； (1分) 简要说明该存储过程所要完成的功能； (2分) 说明该操作涉及操作的表（必须包含两张或两张以上的关系表，以“表名形式”描述） (1分) 表连接涉及字段描述（描述方式为“表 1. 属性=表 2. 属性”） (2分) 该操作会修改字段（以“表名. 字段名”的形式给出），以及修改规则，如新数值的计算方法、在何种条件下予以修改等； (6分) 实现该操作的关键代码（高级语言、SQL），截图即可； (5分) 如何执行该操作，按所述方法能够正常演示程序则给分。
功能描述（1分）	过期活动订单批量结算：一场活动结束后，更新 ticket 表中的该活动下的所有观众从（paid）更新为（Completed）
存储过程功能描述（1分）	首先输入一个 eventid, 之后去 event 表中查询该活动结束时间。如果未结束，则抛出异常；如果已经结束，则通过 innerjoin, 将 ticket 表中活动下状态为 ‘Paid’ 的订单更新为 ‘Completed’，并返回成功更新的行数。
涉及的关系表（2分）	Ticket, Event
表连接涉及	Ticket.EventID = Event.EventID

及 字 段 (1)		
更 改 字 段 (2 分)	字段	规则
	Ticket.OrderStatus	活动已结束且 ticket 状态为 Paid 时，更新字符串为 'Completed'
更 新 代 码 (3 分)	<pre>UPDATE Ticket NATURAL JOIN Event SET Ticket.OrderStatus = 'Completed' WHERE Event.EventID = p_EventID AND Ticket.OrderStatus = 'Paid';</pre>	
创 建 存 储 过 程 源 码 (3 分)	<pre>DELIMITER // CREATE PROCEDURE proc_complete_event_tickets(IN p_EventID INT, OUT p_AffectedRows INT) BEGIN DECLARE v_EndTime DATETIME; SELECT EndTime INTO v_EndTime FROM Event WHERE EventID = p_EventID; -- 判断是否违规 IF v_EndTime > NOW() THEN SIGNAL SQLSTATE '45000' -- 用户自定义异常的状态码 SET MESSAGE_TEXT = '该活动尚未结束'; ELSE UPDATE Ticket NATURAL JOIN Event SET Ticket.OrderStatus = 'Completed' WHERE Event.EventID = p_EventID AND Ticket.OrderStatus = 'Paid'; SET p_AffectedRows = ROW_COUNT(); END IF; END // DELIMITER ;</pre>	

存储过程执行源码（1分）

```
SET @updated_count = 0;
CALL proc_complete_event_tickets(EventID, @updated_count);
SELECT @updated_count AS '成功结算的订单数';
```

高级语言与前端实现：

1. 前端发起：

```
const res = await axios.post(`http://localhost:8080/event/delete?eventId=${eventId}`)
```

2. 后端 control 接口接收：

```
// 结算活动接口
// 前端访问：POST http://localhost:8080/event/complete
@PostMapping("/complete")
public Result<String> completeEventTickets(@RequestParam Integer eventId) {
    try {
        Integer affectedRows = eventService.completeEventTickets(eventId);
        return Result.success("结算成功，共更新了 " + affectedRows + " 条订单状态。");
    } catch (Exception e) {
        // 捕获我们在存储过程里抛出的 SIGNAL 异常（"该活动尚未结束"）
        return Result.error("结算失败：" + e.getMessage());
    }
}
```

3. 后端 server 处理：

```
@Override
public Integer completeEventTickets(Integer eventId) {
    Map<String, Object> params = new HashMap<>();
    params.put("eventId", eventId);

    eventMapper.completeEventTickets(params);

    // 更新了多少条订单
    return (Integer) params.get("affectedRows");
}
```

4. 后端 mapper 调用：

```
// 调用过期活动结算的存储过程
@Select("CALL proc_complete_event_tickets(" +
        "#{eventId, mode=IN, jdbcType=INTEGER}, " +
        "#{affectedRows, mode=OUT, jdbcType=INTEGER})")
@Options(statementType = StatementType.CALLABLE)
void completeEventTickets(Map<String, Object> params);
```

程序演示（2分）



活动管理后台

管理所有活动的上架、下架与状态结算

✓ 结算成功，共更新了 2 条订单状态。

确定

+ 发布新活动

学术讲座管理

大模型前沿技术讲座

2026-04-15 14:00 学术报告厅 余票: 498

结算结束

删除活动

文艺演出管理

十佳歌手总决赛

2026-05-01 19:00 大礼堂 余票: 500

结算结束

删除活动

校园大麦 Campus Damai

大厅首页 我的票夹 发布活动 张三 退出

管理您的所有购票订单

未支付 已支付 已结束

大模型前沿技术讲座

2026-04-15 14:00 学术报告厅

订单编号: #24

已购票

退票 / 删除

极其火爆的讲座

2026-10-01 10:00 大礼堂

订单编号: #23

已购票

退票 / 删除

大模型前沿技术讲座

2026-04-15 14:00 学术报告厅

订单编号: #29

已购票

退票 / 删除

我的票夹

管理您的所有购票订单

未支付 已支付 已结束

大模型前沿技术讲座

2026-04-15 14:00 学术报告厅

已结束

订单编号: #24

待评价

大模型前沿技术讲座

2026-04-15 14:00 学术报告厅

已结束

订单编号: #20

待评价

程序演示（2分）	localhost:5173 显示 我的票夹 发布活动 张三 确定要手动触发该活动的结算吗? (会校验时间是否结束) 确定 取消 结算 文艺演出管理 术报告厅 余票: 498 结算结束 删除活动 十佳歌手总决赛 2026-05-01 19:00 大礼堂 余票: 500 结算结束 删除活动 迎新晚会 2026-09-01 19:00 大礼堂 余票: 10 结算结束 删除活动 礼堂 余票: 0 结算结束 删除活动 礼堂 余票: 50 结算结束 删除活动 未来的跨年演唱会 2099-12-31 20:00 大礼堂 余票: 49 结算结束 删除活动 localhost:5173 显示 结束失败, 活动尚未结束 确定 文艺演出管理 我的票夹 管理您的所有购票订单 未支付 已支付 已结束 未来的跨年演唱会 2099-12-31 20:00 大礼堂 已购票 订单编号: #25 退票 / 删除 极其火爆的讲座 2026-10-01 10:00 大礼堂 已购票 订单编号: #23 退票 / 删除 备注
---------------------	---

7. 含有视图的查询操作（15 分）

说明	（1 分）简要说明该操作所要完成的功能； （1 分）简要说明建立的该视图的功能； （2 分）简要说明该操作涉及的关系数据表（以“表名”的形式给出） （1 分）简要说明表连接涉及的字段（以“表 1. 属性=表 2. 属性”） （6 分）实现该操作的关键代码（高级语言、SQL），截图即可； （4 分）如何执行该操作，按所述方法能够正常演示程序则给分。
操作功能描述 （1 分）	系统首页“活动大厅”的综合数据检索功能。展示活动的标题、时间、主办方名称及场馆具体位置等，而不是像原表 event 中的 id 号。
视图功能描述 （1 分）	将 Event（活动表）、Organizers（主办方表）和 Venue（场馆表）三张表进行自然连接，封装成一个名为 view_event_details 的虚拟表。
涉及的关系表 （2 分）	Event, Organizers, Venue
表连接字段 （1 分）	Event.OrganizerId = Organizers.OrganizerId Event.VenueID = Venue.VenueID
创建视图代码 （3 分）	<pre>CREATE VIEW view_event_details AS SELECT EventID, Title AS EventTitle, StartTime, AvailTix, OrgName AS OrganizerName, Venuename AS VenueName, Location AS VenueLocation FROM Event NATURAL JOIN Organizers NATURAL JOIN Venue;</pre>
查询代码	<pre>SELECT * FROM view_event_details WHERE AvailTix > 0;</pre>

(3分)

高级语言与前端实现:

1. 前端发起:

```
const response = await axios.get('http://localhost:8080/event/list')
```

2. 后端 control 接口接收:

```
// 获取首页活动列表接口
// 前端访问: GET http://localhost:8080/event/list
@GetMapping("/list")
public Result<List<EventDetailsDTO>> getAvailableEvents() {
    List<EventDetailsDTO> list = eventService.getAvailableEvents();
    return Result.success(list);
}
```

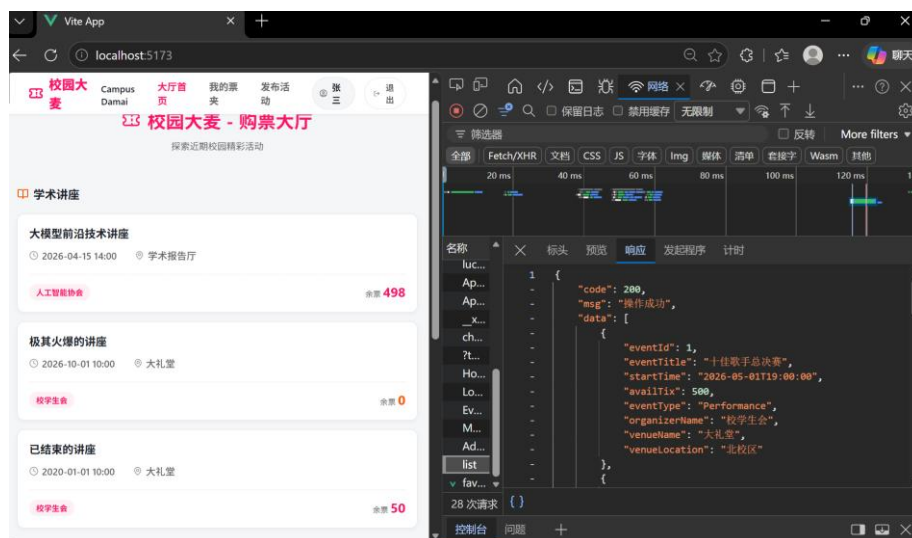
3. 后端 server 处理:

```
@Override
public List<EventDetailsDTO> getAvailableEvents() {
    return eventMapper.getAvailableEventsFromView();
}
```

4. 后端 mapper 调用:

```
@Select("SELECT * FROM view_event_details WHERE AvailTix >= 0")
List<EventDetailsDTO> getAvailableEventsFromView();
```

程序演示
(4分)



备注